



Stiliyan Milanov

Senior Backend Developer

Design Patterns I Actually Use (and When Not To)

Practical .NET Patterns for Maintainable Code

Партньори:

cobuilder

dais
software

d Dynamo
SOFTWARE

Следете актуалните обяви за **.NET**

DEV.BG



About me

- Senior Backend Developer at EGT Digital.
- 7+ years experience with .NET backend systems.
- Background in gaming & fintech.
- Focus on clean code and pragmatic architecture.
- Mentor helping developers grow professionally.



Why bother with design patterns?



- Codebases grow faster than we expect
- Without structure, “God classes” and copy–paste logic appear everywhere.
- Patterns give shared names to solutions and make intent visible in code.

The reality we all know...



Refactor
into
a pattern



Add
another if
/ else if

What this talk is NOT

- ✗ It's not “we do this because someone said it's a good practice”.
- ✗ It's not “I saw it in another project, so let's copy it blindly”.
- ✗ It's not “this pattern is cool, let's force it everywhere”.

What this talk IS

- ✓ Practical patterns I use in real .NET systems.
- ✓ Small before/after examples that show the idea clearly.
- ✓ How each pattern improves clarity and structure.
- ✓ When to use a pattern - and when not to.



Strategy Pattern

Removing branching logic
from your code.



The problem is behavior variation

- One operation, multiple possible behaviors.
- Behavior is selected at runtime.
- Variants evolve independently.

How it looks



Example:

Payment fee calculation

- Each provider has a different fee algorithm.
- Refunds behave differently per provider.
- Rules change independently per provider.
- New providers must be added over time.

When to use Strategy

- Behavior varies by case.
- Each case evolves independently.
- New cases are expected over time.
- Changes should not affect existing behavior.



When NOT to use Strategy

- Logic is simple and stable.
- Only a few cases exist.
- Abstraction adds more complexity than clarity.



Chain of Responsibility

Processing a request
through a sequence of steps



The problem is rigid flow composition

- The order of steps matters.
- Some steps are conditional or may stop the flow.
- The flow must change safely over time.

How it looks





Example: Payment Processing Flow

- Validation, fraud, and compliance vary per payment
- Some checks are skipped, others fail fast
- The order of checks matters for correctness.
- New checks are introduced as regulations evolve.



When to use Chain of Responsibility

- The order of processing matters.
- Steps may be skipped or stop the flow
- You need to add, remove, or reorder steps safely

When NOT to use Chain of Responsibility

- The flow has a few steps.
- The order is fixed and unlikely to change.
- Logic is simple and easy to follow.



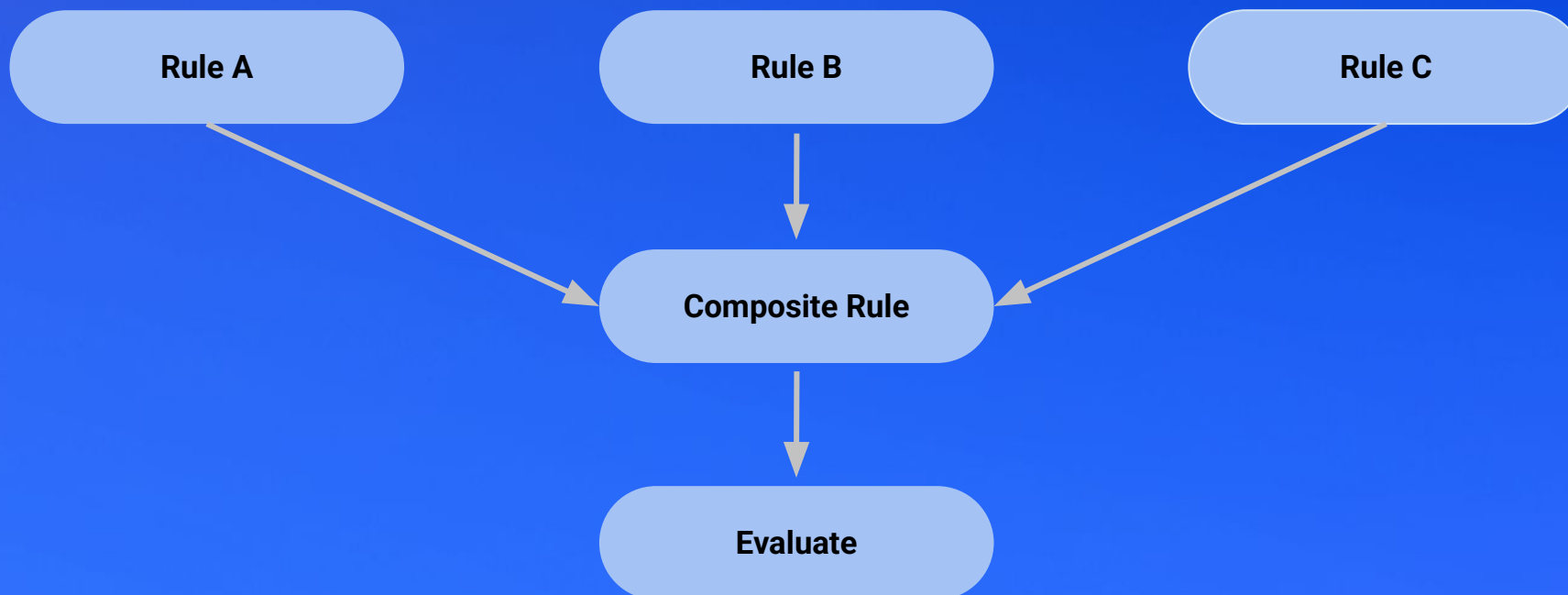
Specification Pattern

Composable business rules
without tangled validation logic

The problem is scattered business rules

- The same rule is implemented in multiple places.
- Rule combinations become messy and fragile.
- Reuse and testing are harder than they should be.

How it looks



Example:

Payment Validation Rules

- A payment is validated by a set of business rules
- Rules are partly shared, partly provider-specific
- Providers differ by limits and constraints
- Rule sets must be composed, not hardcoded



When to use Specification

- Business rules must be reusable.
- Rules need to be combined or composed.
- Validation logic is shared across contexts.
- Rules must be tested independently.

When NOT to use Specification

- Simple one-off checks.
- No reuse or composition needed.
- Rules are always used together.
- Abstraction adds more complexity than clarity.



CQRS

Separate reads and writes
when they have different needs



The problem is mixed responsibilities

- Reads and writes evolve differently.
- Queries want different models than commands.
- One “service” ends up mixing responsibilities.

When CQRS helps

- Complex domain writes + rich querying needs.
- Different performance/scaling needs for reads vs writes.
- You want explicit intent: Command vs Query.
- You need isolated, testable handlers.

When CQRS is overkill

- Simple CRUD app with stable requirements.
- You're adding it "for architecture points".
- It creates more layers than value.
- You don't have real complexity yet.

Patterns are not free

- Every pattern adds indirection.
- Indirection increases cognitive load.
- I use patterns to reduce future change cost.

The Questions I Ask Before Adding a Pattern

- Will this logic change often?
- Will new cases appear?
- Does a small change touch many places?
- Are tests becoming harder to isolate?
- Is the intent of the code getting unclear?

How I decide which pattern to use

Behavior varies by case	Strategy
Sequential steps with early exit	Chain of Responsibility
Reusable business rules	Specification
Reads $\neq$ Writes in complexity	CQRS
Simple, stable logic	No pattern

Thank you!

Contacts:

in <https://www.linkedin.com/in/stiliyan-milanov/>

 stanley9119@gmail.com

